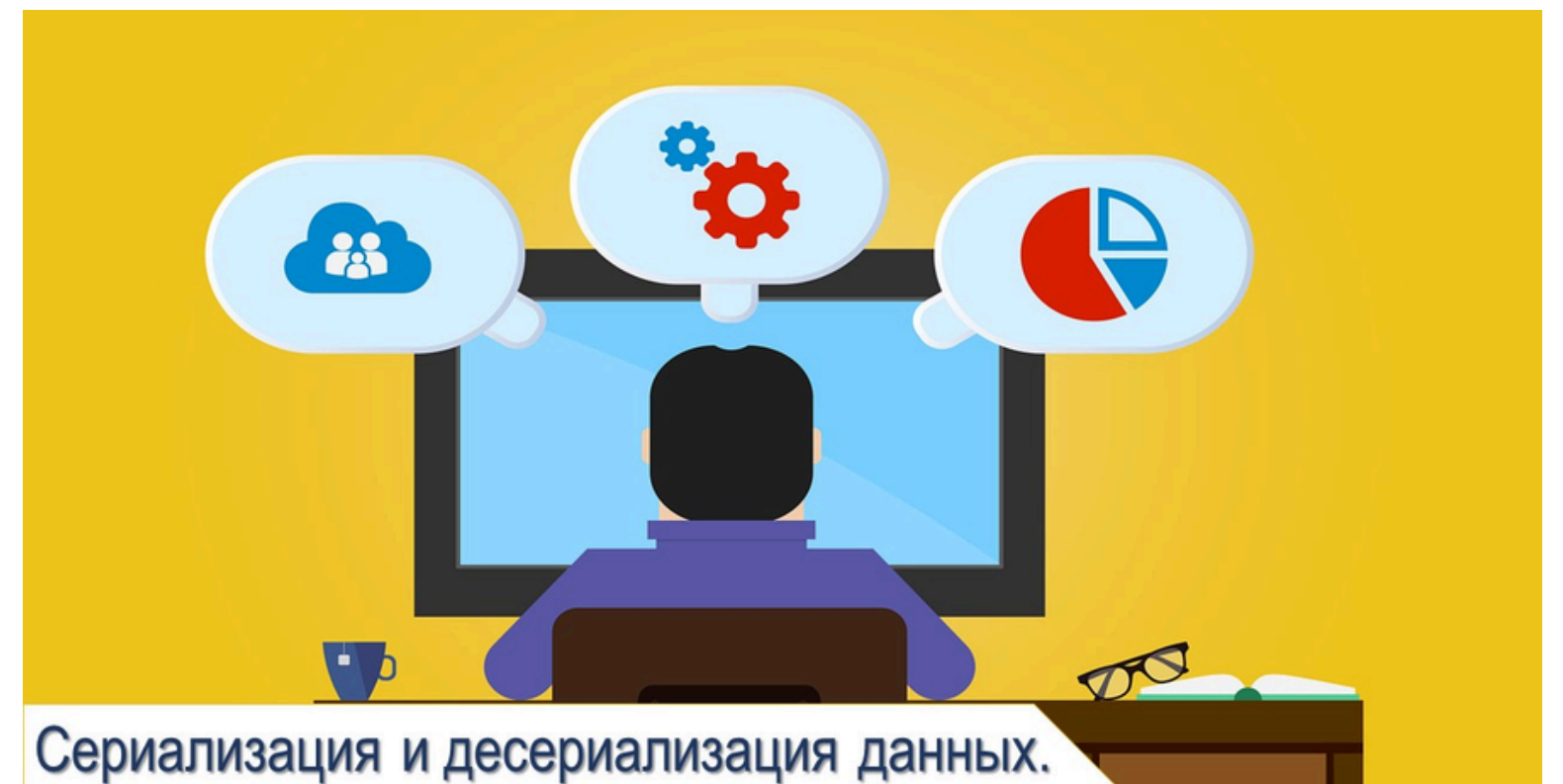


Упаковка Данных



Сериализация и Десериализация



Сериализация и десериализация

Сериализация и десериализация объектов Python является важным аспектом любой нетривиальной программы. Если в Python вы сохраняете что-то в файле, если вы читаете файл конфигурации или отвечаете на HTTP-запрос, вы выполняете сериализацию и десериализацию объектов.

В некотором смысле, сериализация и десериализация - самые скучные вещи в мире. Кто заботится обо всех форматах и протоколах? Вы просто хотите сохранить или стримить некоторые объекты Python и вернуть их позже.

Это очень здоровый способ взглянуть на мир на концептуальном уровне. Но на прагматическом уровне, какая схема сериализации, формат или протокол, которые вы выбираете, могут определять, насколько быстро работает ваша программа, насколько она безопасна, насколько свободно вы должны поддерживать свое состояние и насколько хорошо вы собираетесь взаимодействовать с другими системы.

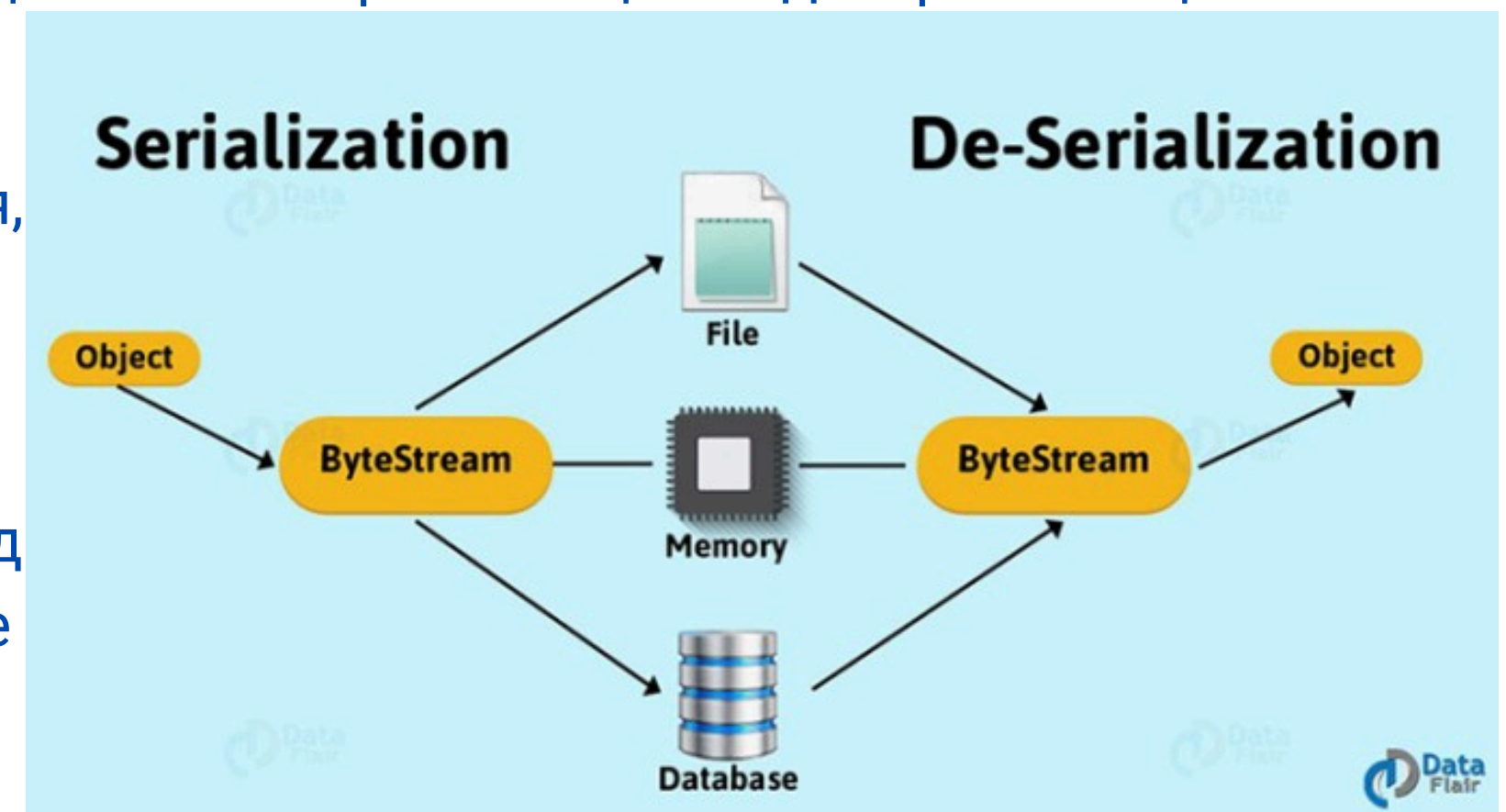
Причина в том, что существует так много вариантов, что разные обстоятельства требуют разных решений. Нет серебряной поли. Далее вы узнаете о преимуществах и недостатках сериализации и десериализации, покажу, как их использовать.

Сериализация определение

Сериализация — это преобразование объекта или дерева объектов в какой-либо формат (обычно текстовый или в набор байт) с тем, чтобы потом исходные объекты можно было восстановить из этого формата. Используется, например, для сохранения состояния программы (то есть, некоторых её объектов) между запусками.

Как следует из определения, сериализация используется для передачи объектов по сети и для сохранения их в файлы. Например, нужно создать распределённое приложение, разные части которого должны обмениваться данными со сложной структурой. В таком случае для типов данных, которые предполагается передавать, пишется код, который осуществляет сериализацию и десериализацию.

Объект заполняется нужными данными, затем вызывается код сериализации, в результате получается, например, XML-документ, JSON. Результат сериализации передаётся принимающей стороне по, скажем, электронной почте или HTTP. Приложение-получатель создаёт объект того же типа и вызывает код десериализации, в результате получая объект с теми же данными, что были в объекте приложения-отправителя.



Применение

Любой из схем сериализации присуще то, что кодирование данных последовательно по определению, и извлечение любой части сериализованной структуры данных требует, чтобы весь объект был считан от начала до конца и воссоздан. Во многих приложениях такая линейность полезна, потому что позволяет использовать простые интерфейсы ввода-вывода общего назначения для сохранения и передачи состояния объекта. В приложениях, где важна высокая производительность, может иметь смысл использовать более сложную, нелинейную, организацию хранения данных.

Сериализация предоставляет несколько полезных возможностей:

- метод реализации сохраняемости объектов, который более удобен, чем запись их свойств в текстовый файл на диск и повторная сборка объектов чтением файлов;
- метод осуществления удалённых вызовов процедур;
- метод распространения объектов, особенно в технологиях компонентно-ориентированного программирования
- метод обнаружения изменений в данных, изменяющихся со временем.

Десериализация определение

Десериализация — это процесс, превращающий специальный код (например, текст в формате JSON) обратно в объекты, с которыми удобно работать в программе. Это важно для сохранения и обмена данными между разными частями программы или даже разными программами.

Десериализация решает проблему восстановления состояния объектов из данных, сохраненных или переданных в упрощенном формате. Это ключ к пониманию и использованию данных, полученных из файлов, баз данных или других программ.

Знание этого процесса упрощает написание программ, позволяя эффективно обмениваться данными и восстанавливать состояния объектов после перезапуска программы или в рамках сетевого взаимодействия. Это основа для работы с распределенными системами, веб-приложениями и многим другим.



Десериализация форматы, действие, безопасность

Выбор формата сериализации и его влияние

Форматы, такие как **сериализация JSON**, **сериализация XML** и бинарные форматы, имеют различное влияние на процесс десериализации. Выбор формата зависит от нескольких факторов:

- **Читаемость:** JSON и XML легче читать и редактировать человеком, в отличие от бинарных форматов.
- **Производительность:** Бинарные форматы обычно обеспечивают более высокую скорость сериализации и десериализации, а также меньший размер данных.
- **Совместимость:** JSON и XML широко поддерживаются различными языками программирования и платформами.

Десериализация в действии

Десериализация играет ключевую роль в **распределенных системах**, **веб-приложениях** и при **обмене данными** между разными программами. Она позволяет:

- Восстанавливать состояния объектов после перезапуска программы.
- Обмениваться данными между клиентом и сервером в веб-приложениях.
- Сохранять и загружать состояние игры, как в приведенном выше примере.

Безопасность десериализации критически важна, поскольку недоверенные данные могут привести к атакам, направленным на внедрение вредоносного кода. Для обеспечения безопасности следует:

- Использовать **проверенные библиотеки** для десериализации.
- Десериализовывать данные **только из доверенных источников**.
- Применять механизмы **валидации** и **санитизации** данных перед десериализацией.

Сериализация и десериализация пример

Пример

Представьте, что вы играете в компьютерную игру, где вы прошли несколько уровней и заработали определенное количество очков. Игра позволяет сохранять ваш прогресс, чтобы вы могли вернуться и продолжить с того места, где остановились. Когда вы сохраняете игру, информация о вашем персонаже, его уровне, инвентаре и достигнутых очках превращается в набор данных, который записывается в файл на вашем компьютере. Этот и есть **сериализация**.

Теперь, когда вы в следующий раз запускаете игру и выбираете "Продолжить", игра загружает этот файл. Затем она **десериализует** данные из файла, восстанавливая состояние игры точно таким, каким оно было в момент сохранения. Ваш персонаж появляется с тем же количеством очков, в том же месте и с теми же предметами в инвентаре.

(я еще думал вставить картинку из Arcanum, но это все таки симпотичнее:)





Преимущества сериализации и десериализации

Подытожим. Технология сериализации и десериализации имеет несколько преимуществ, которые делают ее важным инструментом при работе с данными в программировании:

- **Передача данных по сети** - сериализация позволяет передавать сложные структуры данных, такие как объекты, через сеть в удобном формате. Это особенно полезно при клиент-серверных взаимодействиях и в распределенных системах.
- **Хранение данных** - сериализация позволяет сохранять состояние объектов в файлы или базы данных. Это полезно, когда необходимо сохранять промежуточное состояние программы, чтобы возобновить работу позже.
- **Интеграция разных систем** - множество программ взаимодействуют между собой, и они могут быть написаны на разных языках программирования. Сериализация позволяет преобразовывать данные в общие форматы, такие как JSON или XML, что упрощает интеграцию.
- **Производительность** - в некоторых случаях бинарные форматы сериализации могут быть более компактными и быстрыми для передачи по сети, чем текстовые форматы.
- **Сохранение совместимости** - при использовании версионирования данных, можно сохранить совместимость между разными версиями программ, десериализуя старые данные в новой версии приложения.
- **Расширяемость** - многие форматы сериализации позволяют встраивать пользовательские типы данных, что делает их более гибкими для разных приложений.



Применение и недостатки

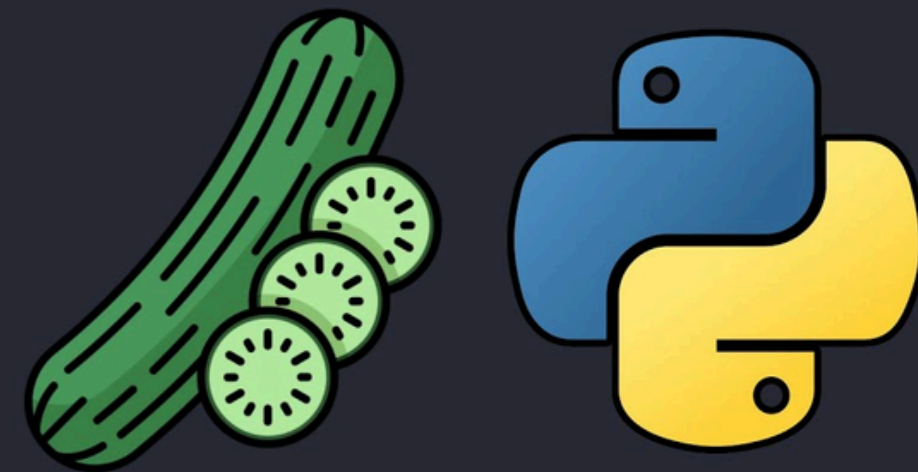
Для наиболее эффективного использования этих возможностей необходимо поддерживать независимость от архитектуры. Например, необходимо иметь возможность надёжно воссоздавать сериализованный поток данных независимо от порядка байтов, используемого в данной архитектуре. Это значит, что наиболее простая и быстрая процедура прямого копирования участка памяти, в котором размещается структура данных, не может работать надёжно для всех архитектур. Сериализация структур данных в архитектурно-независимый формат означает, что не должно возникать проблем из-за различного порядка следования байт, механизмов распределения памяти или различий представления структур данных в языках программирования.

Недостатки

Сериализация нарушает непрозрачность абстрактного типа данных, потенциально раскрывая частные детали реализации. Тривиальные реализации, которые сериализуют все элементы данных, могут нарушать инкапсуляцию.

Чтобы озадачить конкурентов в плане создания аналогичных продуктов, разработчики патентованного программного обеспечения часто держат детали форматов сериализации своих программ в секрете. Некоторые намеренно запутывают или даже шифруют сериализованные данные. Тем не менее, совместимость требует, чтобы приложения могли понимать форматы сериализации друг друга. Поэтому архитектуры удаленного вызова методов, такие как CORBA (обеспечивает взаимодействие между системами, работающими под разными операционными системами, написанными на разных языках программирования и запущенных на разном вычислительном оборудовании), детально задают свои форматы сериализации.

Модуль pickle



СЕРИАЛИЗАЦИЯ ОБЪЕКТОВ

Pickle что это такое?

Модуль **pickle** реализует двоичные протоколы для сериализации и де-сериализации объектной структуры Python.

«Pickling» - это процесс преобразования иерархии объектов Python в поток байтов.

«unpickling» - обратная операция, при которой поток байтов (из binary file или bytes-like object) преобразуется обратно в иерархию объектов.

Пиклинг (и распиклинг) альтернативно называют «сериализацией», «маршалингом», 1 или «сплющиванием»; однако, чтобы избежать путаницы, здесь используются термины «пиклинг» и «распиклинг».

Формат потока данных

Формат данных, используемый pickle, специфичен для Python. Преимуществом этого формата является отсутствие ограничений, налагаемых внешними стандартами, такими как JSON или XDR (которые не могут представлять совместное использование указателей); однако это означает, что не-Python программы не смогут восстановить маринованные Python объекты. По умолчанию формат данных pickle использует относительно компактное двоичное представление.

Pickle протоколы

В настоящее время существует 6 различных протоколов, которые могут быть использованы для маринования. Чем выше используемый протокол, тем более новая версия Python необходима для чтения созданного pickle.

- Протокол версии 0 является оригинальным «человекочитаемым» протоколом и обратно совместим с более ранними версиями Python.
- Протокол версии 1 - это старый двоичный формат, который также совместим с ранними версиями Python.
- Протокол версии 2 был представлен в Python 2.3. Он обеспечивает гораздо более эффективную выборку new-style classes.
- Протокол версии 3 был добавлен в Python 3.0. Он имеет явную поддержку объектов bytes и не может быть распакован Python 2.x. Этот протокол использовался по умолчанию в Python 3.0–3.7.
- Протокол версии 4 был добавлен в Python 3.4. Он добавляет поддержку очень больших объектов, пикинг большего количества типов объектов и некоторые оптимизации формата данных. Начиная с Python 3.8 он является протоколом по умолчанию.
- Протокол версии 5 был добавлен в Python 3.8. Он добавляет поддержку внеполосных данных и увеличивает скорость передачи внутриполосных данных.

Pickle методы и константы

Pickle – является основным продуктом. Это собственный формат сериализации объекта Python.

Интерфейс pickle обеспечивает четыре метода:

- `dump` - сериализует в открытый файл (файл-подобный объект).;
- `dumps` - сериализует в строку.;
- `load` - десериализует из открытого файлового объекта;
- `loads` - десериализует из строки.

Однако, если вы хотите получить больше контроля над сериализацией и де-сериализацией, вы можете создать объект Pickler или Unpickler соответственно.

Модуль pickle предоставляет следующие константы:

- **`pickle.HIGHEST_PROTOCOL`** - целое число, наибольшее из доступных protocol version. Это значение может быть передано в качестве протокольного значения функциям `dump()` и `dumps()`
- **`pickle.DEFAULT_PROTOCOL`** - целое число, по умолчанию protocol version, используемое для травления. Может быть меньше, чем `HIGHEST_PROTOCOL`. В настоящее время по умолчанию используется значение 4, впервые введенное в Python 3.4 и несовместимое с предыдущими версиями.

Pickle что же это такое

Далее в примерах работы с pickle я буду использовать как простой так и сложный объекты Python:

- в качестве простого примера будет использован обычный словарь;
- в качестве сложного экземпляр класса;
- напомним классы для упаковки и распаковки;
- также я покажу как можно отправить свой код в псевдоформате, так чтобы обычный пользователь не смог его просмотреть, а даже если пользователь уже и не совсем обычный, ему придется немного поломать голову;
- ну и наглядно будет показано почему не стоит доверять файлам pickle из неизвестных или непроверенных источников

Pickle практика - начнем с простого:

В качестве простого примера, мы будем использовать словарь с различными типами данных.

```
import pickle

my_data = {
    'nums': [1, 2.0, 3, 4 + 6],
    'strings': ("character_string", b"byte_string"),
    'other': {None, True, False}
}

# для обычной работы
my_data_s = pickle.dumps(my_data, pickle.HIGHEST_PROTOCOL)
print(my_data_s)
my_data_ds = pickle.loads(my_data_s)
print(my_data_ds)
```

```
>>> b'\x80\x05\x95[\x00\x00\x00\x00\x00\x00\x00\x00}\x94(\x8c\x04nums\x94]\x94(K\x01G@\x00\x00\x00\x00\x00\x00\x00K\x03K\ne\x8c\x07strings\x94\x8c\x10character_string\x94C\x0bbyte_string\x94\x86\x94\x8c\x05other\x94\x8f\x94(N\x88\x89\x90u.
{'nums': [1, 2.0, 3, 10], 'strings': ('character_string', b'byte_string'), 'other': {None, True, False}}
```

Как видим по результату выполнения кода после проведения сериализации при помощи `pickle.dumps(my_data, pickle.HIGHEST_PROTOCOL)`, мы получаем байтовую строку. А после проведения десериализации при помощи `pickle.loads(my_data_s)` мы получили обычный словарь который и передавали для сериализации.

```
# для записи в файл
with open('data_pickle.pkl', 'wb') as file:
    pickle.dump(my_data_ds, file, protocol=5)

# для чтения из файла
with open('data_pickle.pkl', 'rb') as file:
    my_data_ff = pickle.load(file)

print(my_data_ff)
```

```
>>> data_packing_pickle
_01_simple.py
data_pickle.pkl
{'nums': [1, 2.0, 3, 10], 'strings': ('character_string', b'byte_string'), 'other': {None, True, False}}
```

Как видим по результату выполнения кода после проведения сериализации мы получили файл формата `.pkl` (мы задаем его самостоятельно это может быть формат и `pickle`) суть в том что если мы откроем полученный файл текстовым редактором. То мы получим нечитаемый набор символов. При десериализации мы получаем тот самый словарь которые у нас был изначально.



Pickle практика - объекты класса:

В качестве простого примера, мы будем использовать двусвязный список с прошлой темы, с прописанными в нем методами, по добавлению узлов с головы и хвоста, а также методы для вывода содержимого нашего списка в терминал.

```
import pickle

class Node:
    def __init__(self, data, next_node=None, prev_node=None):
        self.data = data
        self.next_node = next_node
        self.prev_node = prev_node

class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert_at_head(self, data):
        ...

    def insert_at_tail(self, data):
        ...

    def print_ll_from_head(self):
        ...

    def print_ll_from_tail(self):
        ...
```

```
box_list = LinkedList()
box_list.insert_at_head("Барсик_1")
box_list.insert_at_head("Снежок_0")
box_list.insert_at_tail("Персик_2")

box_list = pickle.dumps(box_list)
print(box_list)

box_list = pickle.loads(box_list)
box_list.print_ll_from_head()
```

Далее мы создаем экземпляр класса список, заполняем его данными, и проводим операцию по сериализации, объекта класса **box_list**. В результате сериализации, мы получаем уже не объект класса а байтовую строку, которая несет информацию об объекте этого класса. До проведения десериализации, мы не сможем применять Методы класса к упакованному (сериализованном, запикленому) объекту.

↓ ↓ ↓

b'\x80\x04\x95\xbe\x00\x00\x00\x00\x00\x00\x00\x8c\x08__main__\x94\x8c\nLinkedList\x94\x93\x94)\x81\x94}\x94(\x8c\x04head\x94h\x00\x8c\x04Node\x94\x93\x94)\x81\x94}\x94(\x8c\x04data\x94\x8c\x0e\xd0\xa1\xd0xbd\xd0\xb5\xd0\xb6\xd0xbe\xd0xba_0\x94\x8c\tnext_node\x94h\x07)\x81\x94}\x94(h\n\x8c\x0e\xd0\x91\xd0\xb0\xd1\x80\xd1\x81\xd0\xb8\xd0xba_1\x94h\x0ch\x07)\x81\x94}\x94(h\n\x8c\x0e\xd0\x9f\xd0\xb5\xd1\x80\xd1\x81\xd0\xb8\xd0xba_2\x94h\x0cN\x8c\tprev_node\x94h\rubh\x13h\x08ubh\x13Nub\x8c\x04tail\x94h\x10ub.'

Снежок_0
Барсик_1
Персик_2

Проводя десериализацию, мы воссоздаем объект класса из байтовой строки, после чего мы опять сможем работать с ним как с объектом. И применять к нему методы которые прописаны в самом классе.

***ВАЖНО! ДЛЯ РАБОТЫ С ОБЪЕКТОМ КЛАССА У ВАС ДОЛЖЕН БЫТЬ НЕОБХОДИМЫЙ КОД.** По похожему принципу работают исполняемые программы в вашей ОС. Для работы нужен не только исполняемый файл, но и файлы для работы программы (дополнительные инструкции, файлы конфигурации, изображения, видео аудио и т.п.)



Pickle практика - объекты класса:

Аналогично можем запаковать объект класса в отдельный файл, чтобы потом переслать его куда вам необходимо, но у человека который будет с ним взаимодействовать должен быть и код на который опирается данный объект.

```
import pickle

class Node:
    def __init__(self, data, next_node=None, prev_node=None):
        self.data = data
        self.next_node = next_node
        self.prev_node = prev_node

class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert_at_head(self, data):
        ...

    def insert_at_tail(self, data):
        ...

    def print_ll_from_head(self):
        ...

    def print_ll_from_tail(self):
        ...
```

```
with open('box_pickled.cats', 'wb') as file:
    pickle.dump(box_list, file)

with open('box_pickled.cats', 'rb') as file:
    box_list_from_file = pickle.load(file)

print(box_list_from_file)
box_list.print_ll_from_tail()
    ↓   ↓   ↓
data_packing_difficult
  _02_difficult.py
  __init__.py
  box_pickled.cats
<__main__.LinkedList object at 0x000001C1258D41A0>
Персик_2
Барсик_1
Снежок_0
```

Я говорил что мы сами можем задавать формат, и в данном случае это формат .cats. По результатам работы кода мы получили файл куда и записан код нашего объекта класса. Если вы хотите кого-то запутать, то можете использовать популярный формат например .pdf, тогда этот файл. Будет пробовать открыться программой для работы с pdf файлами например adobe acrobat или foxit pdf reader. При этом будет получено сообщение что файл поврежден либо этот файл не является файлом формата .pdf. Для обычного пользователя иногда этого бывает достаточно чтобы не копать дальше:)



Pickle практика - класс MyPickler, MyUnPickler:

Если вы хотите получить больший контроль, то вы можете написать собственные классы для пиклинга и анпиклинга данных.

```
import pickle

class MyPickler:

    def __init__(self, protocol=pickle.DEFAULT_PROTOCOL):
        # pickle.HIGHEST_PROTOCOL
        if protocol < 0 or protocol > 5:
            self.protocol = pickle.DEFAULT_PROTOCOL
        elif protocol == 0:
            self.protocol = pickle.HIGHEST_PROTOCOL
        else:
            self.protocol = protocol

    def pickle_data(self, data):
        pickled_data = pickle.dumps(data, self.protocol)
        return pickled_data

    def pickle_file(self, data, filename):
        with open(filename, 'wb') as file:
            pickle.dump(data, file, self.protocol)
        return "Произведен пиклинг в файл"
```

```
import pickle

class MyUnPickler:

    @staticmethod
    def unpickle_data(pickled_data):
        unpickle_data = pickle.loads(pickled_data)
        return unpickle_data

    @staticmethod
    def unpickle_file(pickled_filename):
        with open(pickled_filename, 'rb') as file:
            unpickle_data = pickle.load(file)
        return unpickle_data
```

Вот например как будут выглядеть наши кастомные классы для пиклинга и распиклинга данных. Конечно вы можете добавлять дополнительные методы. Если вам это необходимо.



Pickle практика - класс MyPickler, MyUnPickler:

Импортировав нужные классы, мы вполне можем создать экземпляр класса pickler, куда мы можем передать ту версию протокола которая нам необходима, совершать те же действия что и в предыдущих примерах. Попробуйте запустить этот код у себя и посмотрите что получится. Понятно что ваш импорт будет отличаться от моего.

```
from module_10_data_packing.data_packing_difficult._02_difficult import LinkedList
from module_10_data_packing.Class_Pickler._03_Pickler import MyPickler, MyUnPickler
```

```
my_pickler_5 = MyPickler(protocol=5)
my_pickler_4 = MyPickler(protocol=4)
```

```
box_list = LinkedList()
box_list.insert_at_head("Снежок_1")
box_list.insert_at_head("Мурзик_0")
box_list.insert_at_tail("Персик_2")
```

```
box_list = my_pickler_5.pickle_data(box_list)
box_list = MyUnPickler.unpickle_data(box_list)
box_list.print_ll_from_head()
print()
my_pickler_5.pickle_file(box_list, 'll_pickle.cats')
box_list = MyUnPickler.unpickle_file('ll_pickle.cats')
box_list.print_ll_from_head()
```